Conversation History: Redux Toolkit

👱 User

mandatory common basic flow and syntax to use redux toolkit in any react project.

🤖 Claude

Redux Toolkit: Basic Flow and Syntax

The flow

```
Component → dispatch(action) → Store → Reducer (in slice) → new State → Component re-renders
```

1. A component **dispatches** an **action** (a plain object describing what happened).
2. The **store** passes it to the **reducer** (a function that takes `(state, action)` and updates the state).
3. The store holds the new **state**, and any component reading it through `useSelector` re-renders.

1. Install

bash

```bash
npm install @reduxjs/toolkit react-redux
```

2. Create a slice: `src/features/counter/counterSlice.js`

A **slice** is one feature's state plus the reducers and actions that change it.

js

```js
import { createSlice } from "@reduxjs/toolkit";

const initialState = { value: 0 };

const counterSlice = createSlice({
  name: "counter",            // string: prefix for action types, e.g. "counter/increment"
  initialState,               // the starting state
  reducers: {
    increment: (state) => {
      state.value += 1;       // looks like mutation, but Immer makes it safe
    },
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    },
  },
});

export const { increment, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

Terminology:

- **`createSlice(options)`**: a function that takes an object `{ name: string, initialState: any, reducers: object }` and returns a slice object containing `.actions` and `.reducer`.
- **Case reducer**: each function inside `reducers`. Signature: `(state, action) => void | newState`.
- **Action creator**: the auto-generated function (e.g. `increment`). Calling `increment()` returns an action object: `{ type: "counter/increment", payload: undefined }`.
- **`action.payload`**: the data you pass when calling the action creator. `incrementByAmount(5)` gives `{ type: "counter/incrementByAmount", payload: 5 }`.
- **Immer**: the built-in library that lets you write `state.value += 1` while it produces a new immutable state behind the scenes.

3. Configure the store: `src/app/store.js`

```js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer,   // state.counter is managed by counterReducer
  },
});
```

- **`configureStore(options)`**: takes `{ reducer: object | function }` and returns the **store** object (with `getState()`, `dispatch()`, `subscribe()`). It also sets up Redux DevTools and default middleware automatically.
- The keys of the `reducer` object become the keys of the global state (`state.counter`).

4. Provide the store: `src/main.jsx`

```jsx
import { Provider } from "react-redux";
import { store } from "./app/store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

- **`<Provider>`**: a React component that takes a `store` prop and makes the store available to every component inside it through React Context.

5. Use it in components

```jsx
import { useSelector, useDispatch } from "react-redux";
import { increment, incrementByAmount } from "./features/counter/counterSlice";

function Counter() {
  const count = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increment())}>+1</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>+5</button>
    </div>
  );
}
```

- **`useSelector(selectorFn)`**: a hook. The argument is a **selector** function `(state) => any`. It returns whatever the selector returns and re-renders the component when that value changes.
- **`useDispatch()`**: a hook that takes no arguments and returns the store's `dispatch` function.
- **`dispatch(action)`**: takes an action object and sends it to the store. Returns the same action.

6. Async logic (API calls): `createAsyncThunk`

```js
```

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchTodos = createAsyncThunk(
  "todos/fetchTodos",                        // typePrefix: string
  async (arg, thunkAPI) => {                 // payloadCreator: async function
    const res = await fetch("/api/todos");
    return await res.json();                 // becomes action.payload on success
  }
);

const todosSlice = createSlice({
  name: "todos",
  initialState: { items: [], status: "idle", error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchTodos.pending, (state) => {
        state.status = "loading";
      })
      .addCase(fetchTodos.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.items = action.payload;
      })
      .addCase(fetchTodos.rejected, (state, action) => {
        state.status = "failed";
        state.error = action.error.message;
      });
  },
});

export default todosSlice.reducer;
```

Use it in a component:

```jsx
useEffect(() => {
  dispatch(fetchTodos());
}, [dispatch]);
```

Terminology:

- `createAsyncThunk(typePrefix, payloadCreator)`: takes a `string` and an `async` function, and returns a **thunk action creator**. When dispatched, it automatically emits three actions: `pending`, `fulfilled`, `rejected`.
- `payloadCreator(arg, thunkAPI)`: `arg` is whatever you pass when calling `fetchTodos(arg)`. `thunkAPI` is an object with `dispatch`, `getState`, `rejectWithValue`, and so on. Its return value becomes `action.payload`.
- `extraReducers`: handles actions that were defined outside the slice (like thunk actions). It receives a `builder` object, and `.addCase(actionCreator, reducer)` registers one handler.

Folder structure

```
src/
├── app/
│   └── store.js
├── features/
│   ├── counter/counterSlice.js
│   └── todos/todosSlice.js
├── main.jsx
└── App.jsx
```

Checklist for any project

1. Install `@reduxjs/toolkit` and `react-redux`.
2. Write a slice with `createSlice` for each feature.
3. Register each slice's `.reducer` in `configureStore`.
4. Wrap `<App />` in `<Provider store={store}>`.
5. Read state with `useSelector`, change state with `dispatch(actionCreator())`.
6. Use `createAsyncThunk` plus `extraReducers` for API calls.

If you want, I can show how this connects to your Express/MongoDB back-end, for example fetching data from your API into a slice.

👱 User

d

Claude

Your message came through as just "d", so I'm not sure what you meant to ask. Was it a typo, or did the rest of your message get cut off?

If you were continuing from the Redux Toolkit guide, here are a few things I can help with next:

- Connecting a slice to your Express/MongoDB API
- Adding a second slice (for example, auth or todos) and combining them in the store
- Handling errors with `rejectWithValue` in `createAsyncThunk`
- Using Redux Toolkit with a specific project you're building

Just tell me which one, or send your actual question.

👱 User

Attached file: *excerpt_from_previous_claude_message.txt*

```
const counterSlice = createSlice({
  name: "counter",            // string: prefix for action types, e.g. "counter/increment"
  initialState,               // the starting state
  reducers: {
    increment: (state) => {
      state.value += 1;       // looks like mutation, but Immer makes it safe
    },
```

what is Immer? reducers: { increment: (state) => { state.value += 1; // looks like mutation, but Immer makes it safe },

---

Claude

What is Immer?

Immer is a JavaScript library (a package, `immer`) that lets you write code that looks like it directly changes (mutates) an object, while it actually produces a **new, immutable copy** behind the scenes. Redux Toolkit includes it automatically inside `createSlice` and `createReducer`, so you don't install it yourself.

The problem it solves

Redux requires **immutable updates**: you must never change the existing state object. You must return a **new** object. Redux detects changes by comparing the old state reference with the new one (`oldState !== newState`). If you mutate the old object in place, the reference stays the same and React doesn't know anything changed, so it won't re-render.

Without Immer (plain Redux, manual immutability):

```js
increment: (state) => {
  return { ...state, value: state.value + 1 };  // spread operator copies, then overrides
}
```

For nested state this gets painful:

```js
return {
  ...state,
  user: {
    ...state.user,
    profile: {
      ...state.user.profile,
      name: "Rahim",
    },
  },
};
```

With Immer (Redux Toolkit):

```js
increment: (state) => {
  state.value += 1;
}
```

```js
state.user.profile.name = "Rahim";
```

What each part of your excerpt means

```js
reducers: {
  increment: (state) => {
    state.value += 1;
  },
}
```

- `reducers`: an `object` whose keys become action names and whose values are **case reducer** functions.
- `increment`: a case reducer. Its type is `(state: Draft<State>, action: PayloadAction) => void | State`.
- `(state)`: the parameter. In Redux Toolkit this is not the real state but a **draft** (a temporary proxy copy).
- `state.value += 1`: the **assignment operator** `+=` applied to a property. Because `state` is a draft, this change is recorded rather than applied to the original.

How Immer works internally

1. **Draft creation**: Immer wraps the current state in a **Proxy** (a built-in JavaScript object that intercepts operations like reading and writing properties). This proxy is called the **draft**.
2. **Recording**: when you write `state.value += 1`, the proxy intercepts the write and records it. The original state is untouched.
3. **Finalizing**: when your function ends, Immer looks at what changed and builds a **new state object**. It copies only the changed parts and **reuses** (structurally shares) everything else.

```js
const original = { value: 0, other: { a: 1 } };

// Immer produces:
const next = { value: 1, other: original.other };
// next !== original            (new reference, so React re-renders)
// next.other === original.other (unchanged part is reused, so it's efficient)
```

Two rules inside a case reducer

You can do **either** of these, but **not both** in the same reducer:

```js
```

```
// Option 1: mutate the draft (return nothing)
increment: (state) => {
  state.value += 1;
}

// Option 2: return a brand-new value (don't mutate)
reset: () => {
  return { value: 0 };
}
```

If you mutate the draft **and** return a different value, Immer throws an error.

Important limitations

- This "mutation" style is only safe **inside** `createSlice` / `createReducer` reducers. Outside them (in a component, a normal function, or `useState`), mutating state directly is still a bug.
- Direct mutation of state inside normal React `useState` still does not trigger a re-render.

Summary

Term	Meaning
Immutable update	Creating a new object instead of changing the old one
Draft	A proxy copy of state that you are allowed to "mutate"
Structural sharing	Unchanged parts of state are reused, not copied
Immer	The library that converts your mutation-style code into immutable updates

So the comment in the code means: the line reads like mutation, but Immer ensures the real state is never touched and a correct new state is produced.

👱 User

how does immer look like commonly?

🤖 Claude

How Immer Commonly Looks

In Redux Toolkit, Immer code looks like ordinary "mutating" JavaScript inside case reducers. These are the patterns you'll use most.

1. Changing a property

```
js

setName: (state, action) => {
  state.name = action.payload;
}
```

- **Assignment (`=`)** on a draft property. `action.payload` is the value passed when dispatching.

2. Changing a nested property

```
js

updateCity: (state, action) => {
  state.user.address.city = action.payload;
}
```

- **Dot notation** (property access) works at any depth. No spread operators needed.

3. Adding to an array: `push`

```
js
```

```
addTodo: (state, action) => {
  state.items.push(action.payload);
}
```

- `Array.prototype.push(...items)`: takes one or more elements and returns the new array length. Normally it mutates, but on a draft Immer records it.

4. Updating one item in an array: `find`

```
js

toggleTodo: (state, action) => {
  const todo = state.items.find((t) => t.id === action.payload);
  if (todo) {
    todo.completed = !todo.completed;
  }
}
```

- `Array.prototype.find(callback)`: takes a function returning `boolean` and returns the first matching element or `undefined`. Here the returned item is also a draft, so changing it is safe.
- The `if (todo)` check matters because `find` can return `undefined`.

5. Removing from an array: `filter` (return a new value)

```
js

deleteTodo: (state, action) => {
  state.items = state.items.filter((t) => t.id !== action.payload);
}
```

- `Array.prototype.filter(callback)`: takes a function returning `boolean` and returns a **new array** of the elements that passed.
- You assign the result to `state.items`. This is fine because you are mutating the draft, not returning from the reducer.

Alternative with `splice`:

```
js

const index = state.items.findIndex((t) => t.id === action.payload);
if (index !== -1) state.items.splice(index, 1);
```

- `splice(start, deleteCount)`: removes elements in place and returns the removed elements.

6. Deleting a property

```
js

removeKey: (state, action) => {
  delete state.settings[action.payload];
}
```

- `delete` **operator**: removes a property from an object. It returns `boolean`.

7. Replacing the whole state: `return`

```
js

reset: () => initialState
```

or

```
js
```

```
reset: () => {
  return { items: [], status: "idle" };
}
```

- When you replace everything, you **return** a new value and don't touch the draft.

8. Async status updates in `extraReducers`

```js
.addCase(fetchTodos.pending, (state) => {
  state.status = "loading";
})
.addCase(fetchTodos.fulfilled, (state, action) => {
  state.status = "succeeded";
  state.items = action.payload;
})
```

This is probably the most common Immer-style code you'll write in a real project.

9. Standalone `produce` (outside Redux, less common)

```js
import { produce } from "immer";

const nextState = produce(currentState, (draft) => {
  draft.user.name = "Rahim";
  draft.items.push("new");
});
```

- `produce(baseState, recipe)`: takes the original state and a **recipe** function that receives a `draft`. It returns the new immutable state, and `currentState` stays unchanged.
- Redux Toolkit calls this for you inside every case reducer.

Mistakes to avoid

```js
// ❌ Mutating AND returning a different value
bad: (state) => {
  state.value = 5;
  return { value: 10 };
}

// ❌ Mutating outside a reducer (e.g. in a component)
const count = useSelector((s) => s.counter);
count.value = 10;   // state from useSelector is frozen/real state, not a draft

// ✅ Do one or the other, and only inside reducers
```

Quick rule

Inside `createSlice` reducers, write plain mutation-style code (`=`, `push`, `splice`, `delete`). Anywhere else, never mutate state.

👱 User

Attached file: *excerpt_from_previous_claude_message.txt*

```
3. Configure the store: src/app/store.js
js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer,   // state.counter is managed by counterReducer
  },
});
```

from where did the counterReducer came ? it wasnt in the counterSlice anywhere. import { configureStore } from "@reduxjs/toolkit"; import counterReducer from "../features/counter/counterSlice";

export const store = configureStore({ reducer: { counter: counterReducer, // state.counter is managed by counterReducer }, });

import { createSlice } from "@reduxjs/toolkit";

const initialState = { value: 0 };

const counterSlice = createSlice({ name: "counter", // string: prefix for action types, e.g. "counter/increment" initialState, // the starting state reducers: { increment: (state) => { state.value += 1; // looks like mutation, but Immer makes it safe }, incrementByAmount: (state, action) => { state.value += action.payload; }, }, });

export const { increment, incrementByAmount } = counterSlice.actions; export default counterSlice.reducer;

Claude

Where `counterReducer` Comes From

It comes from the **last line** of `counterSlice.js`:

```
js
export default counterSlice.reducer;
```

Why the name doesn't match

`counterReducer` is not defined anywhere in the slice file. It's just a **name you choose** when importing a **default export**.

```
js
import counterReducer from "../features/counter/counterSlice";
```

- **Default export**: a file can have only one `export default`. The importing file can give it **any name**, because no curly braces are used.
- Here, the default export is `counterSlice.reducer`, so whatever name you write on the import side becomes a label for that function.

These all work identically:

```
js
import counterReducer from "../features/counter/counterSlice";
import counterSliceReducer from "../features/counter/counterSlice";
import xyz from "../features/counter/counterSlice";
```

`counterReducer` is just the conventional, readable choice.

Default export vs named export

In the same file you have both kinds:

```
js
export const { increment, incrementByAmount } = counterSlice.actions;  // named exports
export default counterSlice.reducer;                                    // default export
```

	Named export	Default export
Export syntax	`export const x` / `export { x }`	`export default x`
Import syntax	`import { x } from "..."` (curly braces)	`import anyName from "..."` (no braces)
Name on import	Must match the exported name (unless you use `as`)	Any name you like
How many per file	Many	Only one

So:

- `increment` and `incrementByAmount` need `{ }` and exact names.
- The reducer needs no `{ }` and can be named freely.

Where `.reducer` itself comes from

`counterSlice` is the object returned by `createSlice(...)`. Redux Toolkit builds it with two main properties:

```js
counterSlice.actions   // object of action creators: { increment, incrementByAmount }
counterSlice.reducer   // ONE combined reducer function built from your `reducers` object
```

- **`counterSlice.reducer`**: a function with the signature `(state, action) => newState`. Redux Toolkit generates it automatically by combining all the case reducers (`increment`, `incrementByAmount`) you wrote inside `reducers: { ... }`.

The full chain

```
reducers: { increment, incrementByAmount }     (you write these in createSlice)
        ↓ createSlice combines them
counterSlice.reducer                           (one generated reducer function)
        ↓ export default
import counterReducer from "..."               (you name it on import)
        ↓
configureStore({ reducer: { counter: counterReducer } })
        ↓
state.counter                                  (the slice's state in the store)
```

What the `counter:` key does

```js
reducer: {
  counter: counterReducer,
}
```

- **Key (`counter`)**: becomes the property name in the global state, so you read it with `state.counter.value`.
- **Value (`counterReducer`)**: the function that manages that piece of state.

The key name is also your choice. If you wrote `myCount: counterReducer`, you'd read it with `state.myCount.value` instead.

👱 User

why is configureStore is used inside app?

Claude

Why `configureStore` Is in the `app` Folder

`app/` is just a **folder naming convention**, not a Redux rule. `configureStore` could live in any file. The folder name reflects what the file is responsible for.

The idea behind the structure

```
src/
├── app/
│   └── store.js          ← app-wide setup (the one global store)
├── features/
│   ├── counter/counterSlice.js   ← one feature's state
│   └── todos/todosSlice.js       ← another feature's state
```

- `features/` : holds code that belongs to **one feature** (a slice for counter, a slice for todos, and so on).
- `app/` : holds code that belongs to the **whole application**, not to any single feature.

Why the store counts as app-wide

`configureStore` creates the **store**: the single object that holds the state of every feature.

```js
export const store = configureStore({
  reducer: {
    counter: counterReducer,   // from features/counter
    todos: todosReducer,       // from features/todos
    auth: authReducer,         // from features/auth
  },
});
```

- A **slice** knows only about its own piece of state.
- The **store** is where all slices are **combined** into one global state tree.
- There is only **one store per app**, so it doesn't belong to any single feature.

That is why it sits at the app level, and why `store.js` imports from `features/`, but the feature files never import from `store.js` (mostly).

It is a convention, not a requirement

Both of these work exactly the same:

```
src/app/store.js
src/store.js
src/redux/store.js
src/store/index.js
```

The official Redux Toolkit docs and templates use `app/store.js`, so most projects follow it. Pick one style and stay consistent. What matters is that the same `store` is imported in `main.jsx`:

```jsx
import { store } from "./app/store";   // path must match wherever you put it
```

Quick summary

Folder	Contains	Why
`features/`	Slices (`createSlice`)	Each is one feature's state logic
`app/`	Store (`configureStore`)	Combines all features into one global store

👱 User

why only counter.reducer only imported in store not the counter.slice in the store as well?

Claude

Why Only `counterSlice.reducer` Goes Into the Store

The store only needs the one thing it can **run**: the reducer function. The slice object is a **bundle** of several things, and most of them are not the store's business.

What `counterSlice` actually contains

```js
const counterSlice = createSlice({ ... });
```

`counterSlice` is an **object** with several properties:

```js
counterSlice.name       // "counter"
counterSlice.actions    // { increment, incrementByAmount }  → used by COMPONENTS
counterSlice.reducer    // (state, action) => newState       → used by the STORE
```

Each part has a different consumer:

Property	Type	Who uses it	Why
`.actions`	object of action creators	Components (`dispatch(increment())`)	To describe what happened
`.reducer`	function `(state, action) => newState`	Store (`configureStore`)	To update the state

What `configureStore` expects

```js
configureStore({
  reducer: {
    counter: counterReducer,   // each value MUST be a reducer function
  },
});
```

- The `reducer` option takes an **object whose values are reducer functions.**
- The store calls each function like `counterReducer(currentState, action)`.
- If you passed the whole slice object instead, the store would try to call an **object** as a function and fail, because an object is not callable.

```js
// ❌ Wrong: the slice is an object, not a function
reducer: { counter: counterSlice }

// ✅ Right: the reducer is a function
reducer: { counter: counterSlice.reducer }
```

Why the store doesn't need the actions

The store never creates actions. It only **receives** them when a component calls `dispatch`:

```
Component:  dispatch(increment())
                 ↓  action object { type: "counter/increment" }
Store:      passes the action to every reducer
                 ↓
counterReducer: sees type "counter/increment" → runs your increment case → new state
```

- The **component** needs `increment` (to build the action).

- The **store** needs `counterReducer` (to handle the action).

That's why the slice file has two separate exports:

```js
export const { increment, incrementByAmount } = counterSlice.actions;  // for components
export default counterSlice.reducer;                                    // for the store
```

Summary

- The **slice** is a convenience package that `createSlice` gives you.
- The **store** takes only the **reducer function** out of it.
- The **components** take only the **action creators** out of it.